SL2DT-03: Log Ingest Architecture

> **Series:** SL2DT — Sumo Logic to Dynatrace | **Notebook:** 3 of 11 | **Created:** April 2026 | **Last Updated:** 11/11/2026

Overview

Goal of this step: stand up Dynatrace ingest that produces parity with (or improvement over) the Sumo collector footprint. Design Grail buckets, deploy OneAgent + OTel + OpenPipeline pipelines, convert every Field Extraction Rule into an OpenPipeline processor.

This step is gating. Downstream monitors (SL2DT-05), dashboards (SL2DT-06), and the translation pass (SL2DT-04) all assume logs are flowing into the correct buckets with the expected fields. Don't start those until the ingest layer produces at least 5–10% of final production volume for validation.

> **Metrics are a separate workstream.** This notebook covers the **log** ingest path. If the source account also collects metrics — Telegraf, hosted metrics sources, StatsD, or Prometheus scrape — run **SL2DT-10: Migrating Telegraf-Collected Metrics** alongside this step rather than after cutover.

Table of Contents

1. What You'll Produce
2. Prerequisites & Access
3. Grail Bucket Design
4. `_sourceCategory` Mapping — Implementation
5. Deploy OneAgent for Host Collectors
6. Deploy OTel Collector for Non-OneAgent Sources

7. Configure OpenPipeline Pipelines
8. Convert FERs to OpenPipeline Processors
9. Validate Ingest Parity
10. Step Exit Criteria
11. References

Prerequisites

Requirement	Details
Audience	Platform engineering + ingest team
Dynatrace tenant	Gen3 SaaS, Platform Token with `storage:buckets:write` and `settings:objects:write` scopes (OpenPipeline pipelines, ingest sources and routing are Settings 2.0 objects). Changing **routing** needs `settings:objects:write` on `builtin:openpipeline.logs.routing`, which covers every route in the scope — see §2
Prior reading	SL2DT-02 (taxonomy map + collector inventory + FER inventory); OPMIG-01 (OpenPipeline fundamentals); K8S-01 (if K8s sources)
Tools	Terraform + Monaco for config-as-code (recommended); kubectl for K8s; cURL for Platform Token validation

1. What You'll Produce

Artifact	Purpose
Grail bucket configs	One per retention/IAM tier from taxonomy map
OneAgent rollout config	Hosts auto-instrumented
OTel Collector manifests	Non-OneAgent sources (syslog, HTTP, cloud)
OpenPipeline pipeline configs	Per bucket + per `_sourceCategory` scope
OpenPipeline processors	FER equivalents (one per original FER)
Ingest validation report	Parity check: Sumo volume vs DT volume per scope

Commit to `config/ingest/` as Terraform HCL + Monaco YAML.

2. Prerequisites & Access

This step uses two distinct credential types — they are **not interchangeable**, and mixing them up is a common source of "permission denied" confusion during ingest setup.

Platform Token — query/runtime API calls

Platform Token (prefix `dt0s16.`) with these scopes, for runtime/query-time API calls (bucket read/write of data, log/settings read/write, OpenPipeline configuration through the Settings API):

```
storage:buckets:read, storage:buckets:write
storage:logs:read, storage:logs:write
settings:objects:read, settings:objects:write   (includes OpenPipeline
pipelines, ingest sources, routing)
```

Bucket-scoped IAM policies are **not** in this list: a platform token does not manage IAM. Policies, groups and bindings run on an OAuth client — see SL2DT-07 for the client and SL2DT-08 §4 for the Terraform side.

> **Sources:** Dynatrace Terraform provider — argument reference (Dynatrace GitHub) — *"Platform tokens can't be used for IAM (Account Management) or classic resources."*

OpenPipeline is configured through the Settings API. Log pipelines, ingest sources and routing are the Settings 2.0 schemas `builtin:openpipeline.logs.pipelines`, `builtin:openpipeline.logs.ingest-sources` and `builtin:openpipeline.logs.routing`, governed by `settings:objects:read` / `settings:objects:write`. Routing has no owner-based sharing: *"Owner-based control is available for pipelines and ingest sources, and not routing."* A write grant on the routing schema covers every route in that scope — *"This policy gives users write permissions to all routes for a configuration scope and should therefore be used with caution."* Grant it with a policy such as:

```
ALLOW settings:objects:write WHERE settings:schemaId IN
("builtin:openpipeline.logs.pipelines", "builtin:openpipeline.logs.ingest-
sources", "builtin:openpipeline.logs.routing");
```

> Older guides list `openpipeline:configurations:read` / `openpipeline:configurations:write`. Those scopes belonged to the OpenPipeline Configurations API (`GET` / `POST /platform/openpipeline/v1/configurations/{id}`, deprecated in SaaS 1.327), which reached end of life on June 29, 2026. Don't request them for pipeline or routing changes. The read scope is still the one every endpoint on the current OpenPipeline API reference requires — Preview, Matcher, DQL processor, Technology and **List configurations** (`GET /platform/openpipeline/v1/configurations`, which lists configuration identifiers).
>
> **Sources:** Owner-based access control in OpenPipeline (DT docs), SaaS 1.342 release notes (DT docs) — *"On June 29, 2026, we will deactivate the legacy Configurations API and migrate all environments to the new configuration based on the Settings API."*, List configurations (DT docs) — *"you need a platform token or OAuth access token with openpipeline:configurations:read scope"*

Critical: Platform Token uses `Authorization: Bearer <token>`, not `Api-Token`.

```
export DT_TENANT="https://&lt;env-id&gt;.apps.dynatrace.com"
export DT_TOKEN="dt0s16.XXXXX..."
# Smoke test: a one-row Grail query exercises storage:buckets:read +
storage:logs:read.
curl -s -X POST "$DT_TENANT/platform/storage/query/v1/query:execute" \
  -H "Authorization: Bearer $DT_TOKEN" -H "Content-Type: application/json" \
  -d '{"query": "fetch logs, from:-5m | limit 1"}'
```

Don't verify a platform token with the token lookup endpoint. `/api/v2/apiTokens/lookup` takes a classic access token (`dt0c01.`, `Api-Token` auth); platform tokens have no lookup by secret.

> **Sources:** Upgrade to platform tokens (DT docs) — *"Platform tokens don't support lookup by secret value"*.

OAuth client — Terraform-managed bucket definitions

The `dynatrace_platform_bucket` Terraform resource used later in this notebook is OAuth-client-only — a Platform Token cannot drive it, regardless of the `storage:*` scopes above. Provision an OAuth client with:

```
storage:bucket-definitions:read    (View bucket metadata)
storage:bucket-definitions:write   (Write buckets)
storage:bucket-definitions:delete  (Delete buckets)
```

```
export DT_CLIENT_ID="dt0s02.XXXXX"
export DT_CLIENT_SECRET="dt0s02.XXXXX.YYYYY"
export DT_ACCOUNT_ID="&lt;account-uuid&gt;"
```

The Platform Token scopes above (`storage:buckets:*`) gate query/read access to bucket **data** at runtime — they are a separate concern from OAuth-client-gated bucket **definition** CRUD via Terraform.

> This OAuth-client-only requirement isn't unique to buckets — it applies to every IAM resource too, and **AUTOM-04**'s consolidated resource catalog now documents the full list of OAuth-client-only resources in one place (buckets, IAM group/policy/bindings) alongside the Platform-Token-eligible ones.

Terraform + Monaco

```
# Terraform
terraform init

# Monaco (for bulk config promotion)
monaco --version
```

3. Grail Bucket Design

From the taxonomy map (SL2DT-02 output), define buckets.

Bucket Naming Convention

```
custom_logs_{env}[_{scope}]
```

Examples:
- `custom_logs_prod` — production logs, default
- `custom_logs_prod_db` — production database logs (separate retention)

- `custom_logs_preprod` — pre-production
- `custom_logs_dev` — development
- `audit_logs` — audit-trail **log sources** carried over from Sumo (compliance retention). Dynatrace's own audit events are not stored here — see SL2DT-07 §6.

How buckets are managed

Bucket definitions are not managed through a Settings 2.0 schema — no `builtin:bucket-configuration` schema is documented. Manage them with the Terraform `dynatrace_platform_bucket` resource (OAuth client — see §2) or the Monaco `bucket` type (*"Since Dynatrace Monaco CLI version 2.9.0+, the `bucket` type is supported"*). Both need the `storage:bucket-definitions:*` permissions listed in §2.

> **Sources:** dynatrace_platform_bucket (Terraform provider, Dynatrace GitHub), Monaco configuration type fields (DT docs).

Terraform Example

```
resource "dynatrace_platform_bucket" "custom_logs_prod" {
  name         = "custom_logs_prod"
  display_name = "Production Logs"
  retention    = 90
  table        = "logs"
}

resource "dynatrace_platform_bucket" "custom_logs_prod_db" {
  name         = "custom_logs_prod_db"
  display_name = "Production Database Logs"
  retention    = 180
  table        = "logs"
}

resource "dynatrace_platform_bucket" "audit_logs" {
  name         = "audit_logs"
  display_name = "Audit Trail"
  retention    = 365
  table        = "logs"
}
```

Validate buckets exist

```
// Confirm bucket inventory
//
// Corrected 08/12/2026: the field is `display_name` (snake_case), not
`displayName`, and the
// retention field is `retention_days`. The old cell failed with
FIELD_DOES_NOT_EXIST.
fetch dt.system.buckets
| fields name, display_name, retention_days, dt.system.table, records
| sort name asc
```

4. `_sourceCategory` Mapping — Implementation

Two decisions come out of the taxonomy map (SL2DT-02): bucket routing and attribute preservation. Both get implemented in OpenPipeline.

Pattern A — Bucket routing at ingest

Route on a source identifier (namespace, host group, tag, container label) with a **bucket assignment** in the pipeline's Storage stage. The matcher is a DQL condition — for example:

```
k8s.namespace.name == "prod" or startsWith(log.source, "/var/log/prod/")
```

with target bucket `custom_logs_prod`. §7 shows where this sits in the current Settings shape.

Pattern B — Attribute preservation

Carry the Sumo `_sourceCategory` value in a **customer-owned attribute outside the `dt.*` namespace** — this series uses `sumo.source_category` — so translated queries can still group by it. Set it with a **DQL processor** (§7):

```
fieldsAdd sumo.source_category = concat("prod/api/", k8s.deployment.name)
```

Do **not** reuse `dt.source_entity`. It is a Dynatrace-owned field holding the **entity ID** of the log's source (the semantic dictionary describes it as *"The ID of the entity considered the source of the signal"*), it is already populated on most OneAgent logs, and

OpenPipeline adds it only after processing: *"The following fields are added after the Processing stage when Dynatrace runs its entity detection. You can use them only in stages after the Processing stage"* — `dt.source_entity` is on that list. A processor that writes a category into it does not produce the field the queries expect.

> **Sources:** OpenPipeline limits (DT docs). **Dictionary:** `dt.source_entity` (`deprecated`), read 10/05/2026.

Most customers do both: bucket for isolation, attribute for query-time grouping.

Query what you get

After OpenPipeline is live, this should return your taxonomy:

```
// Inspect the preserved _sourceCategory equivalent
fetch logs, from:-1h, bucket:{"custom_logs_prod"}
| summarize c = count(), by:{sumo.source_category}
| sort c desc
| limit 50
```

5. Deploy OneAgent for Host Collectors

Every Sumo Installed Collector has a OneAgent equivalent. For hosts already running a Sumo Installed Collector:

Linux / Windows Hosts

```
# Fetch installer from tenant UI or API, then:
/bin/sh Dynatrace-OneAgent-Linux-1.xxx.sh \
  --set-app-log-content-access=true \
  --set-host-group=prod-web

# Verify
oneagentctl --get-host-tag
oneagentctl --get-logmonitoring-enabled
```

AWS / Azure / GCP Hosts

VMs that ran a Sumo Installed Collector still need OneAgent — deploy it with your provisioning tooling (SL2DT-08 §7). The Clouds app (AWS) and the Azure / GCP integrations add the managed-service metrics and logs; they do not collect host log files or processes, so they complement OneAgent rather than replace it.

Kubernetes

Install the Dynatrace Operator, then configure DynaKube:

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;env-id&gt;.live.dynatrace.com/api
  tokens: tokens-secret
  oneAgent:
    cloudNativeFullStack:
      tolerations: []
      nodeSelector: {}
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
  logMonitoring:
    ingestRuleMatchers:
      - attribute: k8s.namespace.name
        values: ["prod-payments", "prod-web", "preprod-payments"]
```

`apiUrl` is the environment URL *"including the `/api` path at the end"* — on SaaS the `live.dynatrace.com` host, not the `apps.dynatrace.com` platform host. An `ingestRuleMatchers` entry has only `attribute` and `values` (there is no `operator`), and the documentation's examples list exact names — list each namespace rather than relying on wildcards. Plan the list up front: *"This field is immutable once set."*

> **Sources:** DynaKube parameters (DT docs).

Capture host groupings

Tag hosts per team/environment. These tags feed the bucket routing in OpenPipeline.

```
oneagentctl --set-host-tag="team=payments"
oneagentctl --set-host-tag="env=prod"
```

Verify ingest

```
// Confirm OneAgent-instrumented hosts are sending logs
fetch logs, from:-15m
| filter isNotNull(dt.entity.host)
| summarize c = count(), by:{entityName(dt.entity.host)}
| sort c desc
| limit 20
```

6. Deploy OTel Collector for Non-OneAgent Sources

For sources where OneAgent doesn't apply (syslog listener, HTTP upload, message-queue consumer, proprietary agents), deploy an OTel Collector.

Minimal OTel Collector Config

```
receivers:
  syslog:
    tcp:
      listen_address: 0.0.0.0:5514
    protocol: rfc5424
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  resourcedetection:
    detectors: [env, system]
  batch:
    send_batch_size: 1000
    timeout: 5s

exporters:
  otlphttp:
    endpoint: https://&lt;env-id&gt;.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: "Api-Token dt0c01.XXX"

service:
  pipelines:
    logs:
      receivers: [syslog, otlp]
      processors: [resourcedetection, batch]
      exporters: [otlphttp]
```

Note on auth: the exporter above uses a Classic API token (`dt0c01.` , `Api-Token` scheme) with the `logs.ingest` scope, which covers the OpenTelemetry log ingest API. From **SaaS 1.343** (staged tenant rollout from 07/14/2026) platform tokens also authenticate against classic and ingest endpoints: send `Authorization: Bearer dt0s16.…` with the scope that matches the signal — `openpipeline:logs:ingest` for logs, `openpipeline:traces:ingest` for OTLP traces. Confirm 1.343 has reached your tenant before switching; until it has, the `dt0c01` token remains the working path, and it stays supported afterwards.

> **Sources:**
>
> - What's new in Dynatrace SaaS 1.343 (DT docs) — *"You can now use platform tokens to*

> *authenticate against all classic and ingest API endpoints, including logs, metrics, and business events.", "Existing API token authentication remains fully supported, so no changes are required for current integrations."*
>
> ~ Dynatrace API authentication (DT docs) — *"Platform tokens work across ingest endpoints and require a scope that matches the signal type"*; `logs.ingest` *"Grants access to the POST ingest logs request of the Log Monitoring API v2 as well as the OpenTelemetry log ingest API."*

HTTP Source Equivalent

Sumo "Hosted HTTP Source" → OpenPipeline HTTP endpoint (no OTel in between):

```
curl -X POST "https://&lt;env-id&gt;.live.dynatrace.com/api/v2/logs/ingest" \
  -H "Authorization: Api-Token dt0c01.XXX" \
  -H "Content-Type: application/json" \
  -d '[{"content": "app log line", "log.source": "prod/api/web"}]'
```

The log ingest endpoint is on the environment's `live.dynatrace.com` host, as in the documented example — not the `apps.dynatrace.com` host held in `$DT_TENANT` for platform API calls. With a platform token on SaaS 1.343+, replace the header with `Authorization: Bearer <platform-token>` (scope `openpipeline:logs:ingest`).

Scripted ingestion from CloudWatch, EventHub, Pub/Sub — replace with Dynatrace cloud integrations (AWS Clouds app, Azure, GCP) rather than dual-writing via HTTP.

7. Configure OpenPipeline Pipelines

For each bucket, define a pipeline that:

1. **Matches** — which logs belong here
2. **Processes** — parse, enrich, redact
3. **Routes** — to the target bucket

Example: Production API Logs Pipeline

Note on API surface: the OpenPipeline **Configurations API** (the older `pipelineId` / `matchers` / `processors` JSON shape) is deprecated with an end-of-life of **June 29, 2026**. Current pipeline configuration goes through the **Settings API**, using the per-scope schema family `builtin:openpipeline.<scope>.pipelines` (plus `.routing` and

`.ingest-sources` for matching/routing and source config), or the equivalent Terraform resource. Both forms below reflect the current shape.

Settings API — `builtin:openpipeline.logs.pipelines`:

```
{
  "schemaId": "builtin:openpipeline.logs.pipelines",
  "scope": "environment",
  "value": {
    "displayName": "Production API Logs",
    "customId": "pipeline_prod_api",
    "processing": {
      "processors": [
        {
          "type": "dql",
          "id": "processor_parse_http_fields",
          "description": "Parse method/path/status from content",
          "matcher": "true",
          "dql": {
            "script": "parse content, \"LD? 'method=' WORD:http.method '
path=' DATA:http.path ' status=' INT:http.status\""
          },
          "enabled": true
        },
        {
          "type": "dql",
          "id": "processor_add_source_category",
          "description": "Derive sumo.source_category from
k8s.deployment.name",
          "matcher": "true",
          "dql": {
            "script": "fieldsAdd sumo.source_category = concat(\"prod/api/\",
k8s.deployment.name)"
          },
          "enabled": true
        }
      ]
    }
  }
}
```

Terraform equivalent — `dynatrace_openpipeline_v2_logs_pipelines`:

```
resource "dynatrace_openpipeline_v2_logs_pipelines" "pipeline_prod_api" {
  display_name = "Production API Logs"
  custom_id    = "pipeline_prod_api"
  processing {
    processors {
      processor {
        type        = "dql"
        id          = "processor_parse_http_fields"
        description = "Parse method/path/status from content"
        matcher     = "true"
        dql {
          script = "parse content, \"LD? 'method=' WORD:http.method ' path='
DATA:http.path ' status=' INT:http.status\""
        }
        enabled = true
      }
      processor {
        type        = "dql"
        id          = "processor_add_source_category"
        description = "Derive sumo.source_category from k8s.deployment.name"
        matcher     = "true"
        dql {
          script = "fieldsAdd sumo.source_category = concat(\"prod/api/\",
k8s.deployment.name)"
        }
        enabled = true
      }
    }
  }
}
```

Why a DQL processor for the category. The **Add fields** processor writes static values only — *"This processor doesn't leverage DQL processing statements and doesn't support dynamic values."* … *"To add fields with dynamic values, use the DQL processor instead."* Given `concat(...)`, it would store that text literally. The parse pattern starts with `LD?` rather than `LD` because `LD` needs at least one character before `method=`, so it returns nothing for a line that *starts* with `method=` — a line Sumo's `"method=*"` does match.

> **Sources:** OpenPipeline processing stage (DT docs).

Redaction (for example, masking card numbers) belongs in a dedicated masking/DQL processor stage — see OPLOGS-03 and OPIPE for the current redaction processor

shape rather than a standalone `"type": "redact"` block, which does not exist in the current API.

Route-to-bucket is a separate concern from the pipeline's processors — it's handled by the pipeline's `storage` stage / bucket-assignment processor (`bucket_assignment_processor` in Terraform), not a top-level `targetBucket` field on the pipeline object.

Validation Queries

```
// Check that parsing produces the expected fields
fetch logs, from:-15m, bucket:{"custom_logs_prod"}
| filter sumo.source_category == "prod/api/payments"
| filter isNotNull(http.status)
| summarize c = count(), by:{http.method, http.status}
| sort c desc
```

8. Convert FERs to OpenPipeline Processors

Every Sumo FER becomes one OpenPipeline parse processor. The logic is the same; only the pattern syntax changes.

Sumo FER Example

```
# FER name: api-parse-req
# Scope: _sourceCategory=prod/api/*
# Parse expression:
"method=*" "path=*" "status=*"
```

Fields extracted: `method`, `path`, `status`.

Dynatrace OpenPipeline Equivalent

A DQL processor, in the same Settings shape as §7, on the pipeline whose route covers the FER's scope (`prod/api/*`) — the route does the scoping, so the processor matches everything that reaches it:

```
{
  "type": "dql",
  "id": "processor_fer_api_parse_req",
  "description": "FER api-parse-req",
  "matcher": "true",
  "dql": {
    "script": "parse content, \"LD? 'method=' WORD:http.method ' path='
DATA:http.path ' status=' INT:http.status\""
  },
  "enabled": true
}
```

DPL Pattern Cheat Sheet

Sumo regex	DPL matcher
`.+` (greedy)	`DATA` (lazy, default)
`\w+` (word chars)	`WORD`
`\d+` (digits)	`INT`
`\d+\.\d+` (float)	`DOUBLE`
`\S+` (non-space)	`NSPACE` (`LD` matches any characters, spaces included)
`\d+\.\d+\.\d+\.\d+` (IPv4)	`IPADDR`
ISO 8601 timestamp	`ISO8601` (or `TIMESTAMP('<format>')` with an explicit format)
Quoted string	`DQS` (double-quoted) / `SQS` (single-quoted)
`\{.*\}` (JSON blob)	`JSON`

Every row was checked with `data record(content="…") | parse content, "…"` on 10/05/2026. **FAQ-15** is the corpus's full DPL reference, including the matchers that do not exist.

FER Conversion Workflow

1. For each FER in `inventory/fers.json`, identify the scope.
2. Map the parse expression to DPL matchers.

3. Create the OpenPipeline processor on the matching pipeline.
4. Validate extraction via a DQL query.

Check: Parsed fields are available

```
// After FER conversion, confirm extraction is working
fetch logs, from:-15m, bucket:{"custom_logs_prod"}
| filter startsWith(sumo.source_category, "prod/api/")
| filter isNotNull(http.method)
| limit 10
| fields content, http.method, http.path, http.status
```

9. Validate Ingest Parity

Parity = volume per scope per hour in DT is within 5% of Sumo volume for the same scope.

Sumo-side baseline

```
*
| count by _sourceCategory
| where _count &gt; 0
```

Dynatrace-side comparison

```
// Count per scope in DT — compare to Sumo baseline
fetch logs, from:-1h
| summarize c = count(), by:{sumo.source_category}
| sort c desc
| limit 50
```

Parity report

`_sourceCategory`	Sumo count/hr	DT count/hr	% diff	Status
prod/api/payments	18,452	18,110	-1.9%	✅
prod/web/frontend	9,300	9,044	-2.8%	✅
prod/db/primary	2,103	1,845	-12.3%	⚠️ investigate
...	...	...	...	...

Target: ≥95% of scopes within 5% diff. Flag any scope with >10% diff for investigation before proceeding.

Common Parity Gaps

Symptom	Likely Cause	Fix
DT count much lower	OneAgent or OTel not deployed to all sources	Check deployment inventory
DT count much higher	Duplicate ingest paths (Sumo still writing + DT ingesting same source twice)	Audit collector config
DT missing entire scope	`_sourceCategory` pattern didn't match any OpenPipeline matcher	Adjust matcher
Parsed fields missing	FER processor not applied to the pipeline	Re-check processor scope

10. Step Exit Criteria

G3 — Ingest Parity Achieved

- [ ] All planned buckets exist and retention configured
- [ ] OneAgent deployed to 100% of host-based collectors
- [ ] OTel Collector deployed for non-OneAgent sources
- [ ] OpenPipeline pipelines configured per bucket
- [ ] Every FER has a corresponding OpenPipeline processor
- [ ] `sumo.source_category` populated on all logs routed from Sumo-equivalent sources

- [ ] Parity report: ≥95% of scopes within 5% of Sumo volume
- [ ] No scope with >10% volume gap uninvestigated

Next step: SL2DT-04 — SumoQL → DQL Translation (translate the query surface with the `sumoql-to-dql` skill).

---

11. References

Log ingest paths

- Ingest from (DT docs)
- Dynatrace OneAgent (DT docs)
- OpenTelemetry ingest (DT docs)
- Logs (DT docs)
- Dynatrace API (DT docs)

Routing, parsing, and storage

- OpenPipeline (DT docs)
- DPL Pattern Reference (DT docs)
- Grail (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace or Sumo Logic. Always verify information against the official Dynatrace documentation and Sumo Logic documentation.